Solution: Ransom Note (easy)

Problem Statement

Given two strings, one representing a ransom note and the another representing the available letters from a magazine, determine if it's possible to construct the ransom note using only the letters from the magazine. Each letter from the magazine can be used only once.

Examples:

1. **Example 1**:
    - Input: Ransom Note = "hello", Magazine = "hellworld"
    - Expected Output: true
    - Justification: The word "hello" can be constructed from the letters in "hellworld".
2. **Example 2**:
    - Input: Ransom Note = "notes", Magazine = "stoned"
    - Expected Output: true
    - Justification: The word "notes" can be fully constructed from "stoned" from its first 5 letters.
3. **Example 3**:
    - Input: Ransom Note = "apple", Magazine = "pale"
    - Expected Output: false
    - Justification: The word "apple" cannot be constructed from "pale" as we are missing one 'p'.

Solution

To solve this problem, we will utilize a hashmap to keep track of the frequency of each character in the magazine. First, we iterate through the magazine, updating the hashmap with the count of each character. Then, we go through the ransom note. For each character in the note, we check if it exists in the hashmap and if its count is greater than zero. If it is, we decrease the count in the hashmap, indicating that we've used that letter. If at any point we find a character in the note that isn't available in sufficient quantity in the magazine, we return false. If we successfully go through the entire note without this issue, we return true, indicating the note can be constructed from the magazine.

1. **Populate Frequency Map**: Traverse the magazine string and populate a hashmap with the frequency of each character.
2. **Check Feasibility**: Traverse the ransom note string. For each character, check its frequency in the hashmap. If the character is not present or its frequency is zero, return false. Otherwise, decrement the frequency of the character in the hashmap.
3. **Return Result**: If we successfully traverse the ransom note without returning false, then it's possible to construct the ransom note from the magazine. Return true.

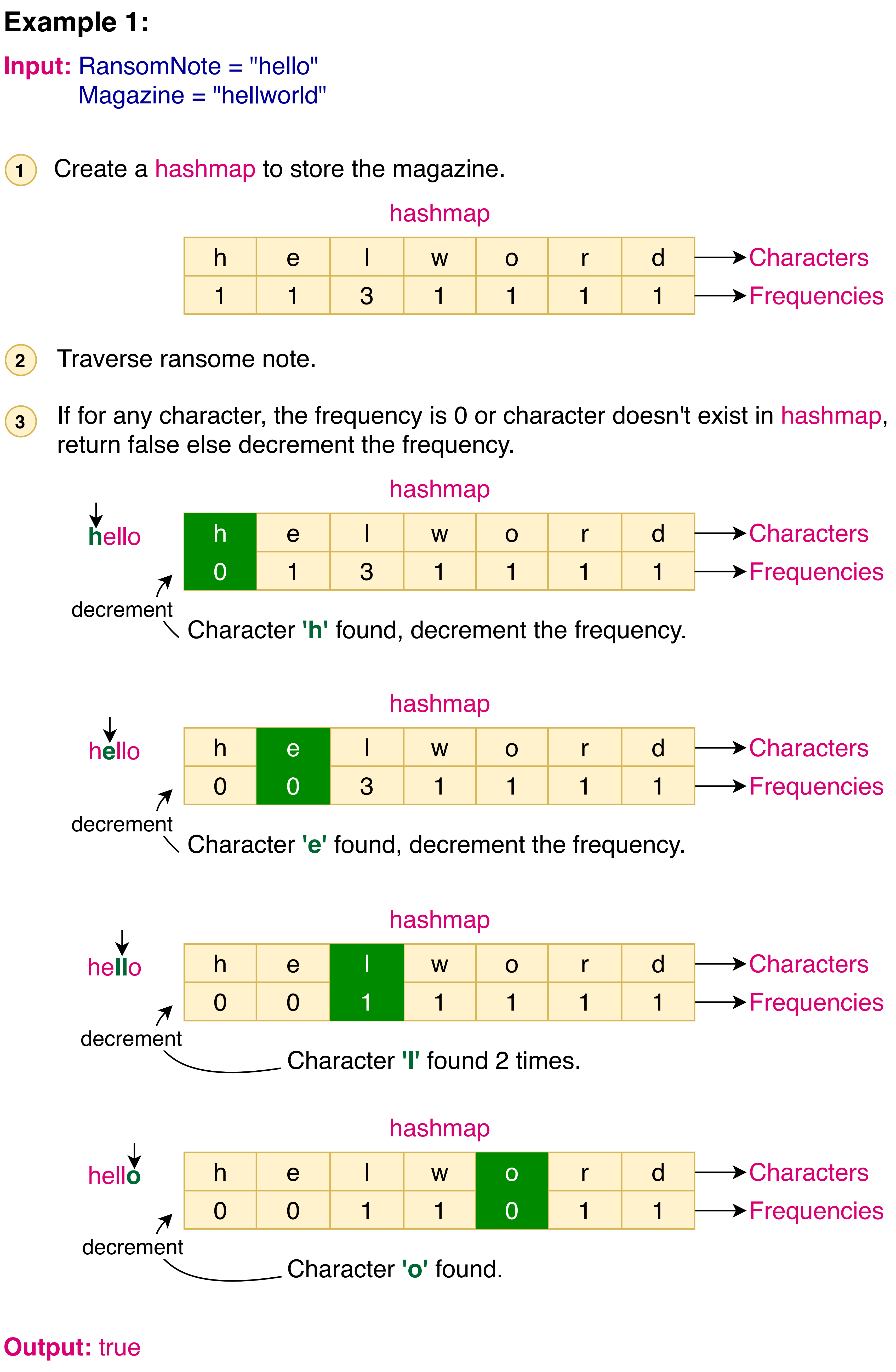
Using a hashmap allows for efficient storage and retrieval of character frequencies, ensuring that we can determine the feasibility of constructing the ransom note in linear time.

Algorithm Walkthrough:

Given the ransom note "hello" and the magazine "hellworld":

- Initialize an empty hashmap.
- Traverse the magazine "hellworld" and populate the hashmap with character frequencies: {'h':1, 'e':1, 'l':3, 'w':1, 'o':1, 'r':1, 'd':1}.
- Traverse the ransom note "hello". For each character:
    - Check its frequency in the hashmap.
    - If the frequency is zero or the character is not present, return false.
    - Otherwise, decrement the frequency of the character in the hashmap.
- Since we can traverse the entire ransom note without returning false, return true.

Here is the visual representation of the algorithm:



Example 1:

Input: RansomNote = "hello"
Magazine = "hellworld"

① Create a hashmap to store the magazine.

hashmap

h	e	l	w	o	r	d
1	1	3	1	1	1	1

② Traverse ransome note.

③ If for any character, the frequency is 0 or character doesn't exist in hashmap, return false else decrement the frequency.

hashmap

h	e	l	w	o	r	d
0	1	3	1	1	1	1

Character 'h' found, decrement the frequency.

hashmap

h	e	l	w	o	r	d
0	0	3	1	1	1	1

Character 'e' found, decrement the frequency.

hashmap

h	e	l	w	o	r	d
0	0	1	1	1	1	1

Character 'l' found 2 times.

hashmap

h	e	l	w	o	r	d
0	0	1	1	0	1	1

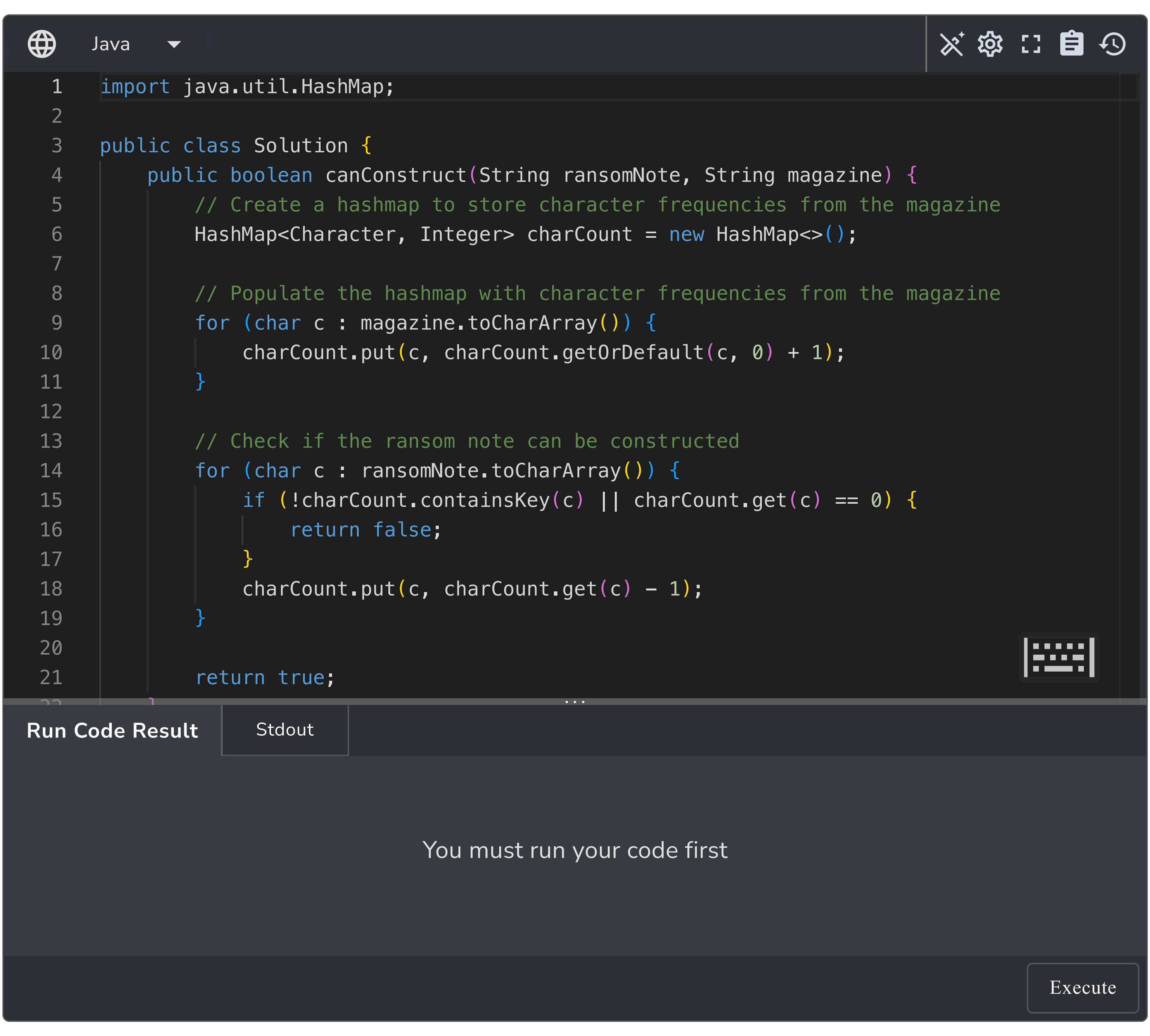
Character 'o' found.

Output: true

Ransom Note

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;

public class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        // Create a hashmap to store character frequencies from the magazine
        HashMap<Character, Integer> charCount = new HashMap<>();

        // Populate the hashmap with character frequencies from the magazine
        for (char c : magazine.toCharArray()) {
            charCount.put(c, charCount.getOrDefault(c, 0) + 1);
        }

        // Check if the ransom note can be constructed
        for (char c : ransomNote.toCharArray()) {
            if (!charCount.containsKey(c) || charCount.get(c) == 0) {
                return false;
            }
            charCount.put(c, charCount.get(c) - 1);
        }

        return true;
```

You must run your code first

Complexity Analysis

Time Complexity: The algorithm traverses both the ransom note and the magazine once, making the time complexity O(n + m), where n is the length of the ransom note and m is the length of the magazine.

Space Complexity: The space complexity is determined by the hashmap, which in the worst case will have an entry for each unique character in the magazine. However, since the English alphabet has a fixed number of characters, the space complexity is O(1).

← Previous
Problem 5: Ransom Note (Easy)

Next →
Introduction To Tree Breadth First Search Pattern

☐ Mark as Completed